

1. Create a NumPy array of integers from 10 to 50.

```
import numpy as np
```

```
arr1 = np.arange(10, 51)
```

2. Create an array of 10 zeros and replace the fifth value with 1.

```
arr1= np.zeros(10)
```

```
arr1[4] = 1
```

3. Create a 3x3 matrix with values from 1 to 9.

```
arr1 = np.arange(1, 10).reshape(3, 3)
```

4. Create a NumPy array of 20 random integers between 1 and 100.

```
arr1 = np.random.randint(1, 101, 20)
```

5. Create a 5x5 identity matrix.

```
identity = np.eye(5)
```

6. Create a NumPy array filled with 7s of shape (4, 4).

```
arr1 = np.full((4, 4), 7)
```

7. Generate a 1D array of 20 linearly spaced elements between 0 and 1.

```
arr1 = np.linspace(0, 1, 20)
```

8. Create a 3x3 matrix with random float values between 0 and 1.

```
arr1 = np.random.rand(3, 3)
```

9. Reverse a given 1D NumPy array.

```
arr1 = np.array([1, 2, 3, 4, 5])[::-1]
```

10. Reshape a 1D array of 12 elements into a 3x4 matrix.

```
arr1= np.arange(12).reshape(3, 4)
```

11. Extract the second column from a 4x4 matrix.

```
matrix11 = np.arange(16).reshape(4, 4)
```

```
second_col = matrix11[:, 1]
```

12. Slice a 5x5 matrix to get the central 3x3 matrix.

```
matrix12 = np.arange(25).reshape(5, 5)
```

```
central = matrix12[1:4, 1:4]
```

13. Replace all odd numbers in an array with -1.

```
arr1 = np.arange(10)
```

```
arr1[arr1 % 2 == 1] = -1
```

14. Get all even numbers from a NumPy array.

```
arr1 = np.arange(20)
```

```
evens = arr1[arr1% 2 == 0]
```

15. Flatten a 2D matrix into a 1D array.

```
arr1 = np.array([[1, 2], [3, 4]])
```

```
flattened = arr1.flatten()
```

16. Find the mean, median, and standard deviation of a NumPy array.

```
arr1 = np.random.randint(1, 100, 20)
```

```
mean = np.mean(arr1)
```

```
median = np.median(arr1)
```

```
std = np.std(arr16)
```

17. Create two arrays and perform element-wise addition, subtraction, multiplication, and division.

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
add = a + b
```

```
sub = a - b
```

```
mul = a * b
```

```
div = a / b
```

18. Multiply two matrices using NumPy dot product.

```
mat1 = np.array([[1, 2], [3, 4]])
```

```
mat2 = np.array([[5, 6], [7, 8]])
```

```
dot_product = np.dot(mat1, mat2)
```

19. Find the minimum and maximum values in a NumPy array and their indices.

```
arr1 = np.random.randint(1, 100, 10)
```

```
min_val = np.min(arr1)
```

```
max_val = np.max(arr1)
```

```
min_idx = np.argmin(arr1)
```

```
max_idx = np.argmax(arr1)
```

20. Normalize a NumPy array (rescale to 0–1).

```
arr1 = np.random.randint(0, 100, 10)
```

```
normalized = (arr1 - arr1.min()) / (arr1.max() - arr1.min())
```

21. Stack two 1D arrays vertically and horizontally.

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
vstack = np.vstack((x, y))
hstack = np.hstack((x, y))
```

22. Replace all elements greater than 10 in a NumPy array with 10.

```
arr1= np.random.randint(1, 20, 10)
arr1 [arr1> 10] = 10
```

23. Broadcast a 1D array to a 2D array of shape (3, 5).

```
a = np.array([1, 2, 3, 4, 5])
broadcasted = np.tile(a, (3, 1))
```

24. Split a 1D array into 4 equal parts.

```
arr1= np.arange(20)
splits = np.array_split(arr1, 4)
```

25. Sort a NumPy array by column or row.

```
arr1 = np.random.randint(1, 50, (4, 4))
row_sorted = np.sort(arr1, axis=1)
col_sorted = np.sort(arr1, axis=0)
```

26. Create a checkerboard pattern (8x8) using 0s and 1s.

```
checkerboard = np.zeros((8, 8), dtype=int)
checkerboard[1::2, ::2] = 1
checkerboard[::2, 1::2] = 1
```

27. Create a border of 1s around a 0 matrix (e.g., 5x5).

```
border = np.ones((5, 5), dtype=int)
```

```
border[1:-1, 1:-1] = 0
```

28. Find the common values between two arrays.

```
a = np.array([1, 2, 3, 4])
```

```
b = np.array([3, 4, 5, 6])
```

```
common = np.intersect1d(a, b)
```

29. Remove duplicate values from a NumPy array.

```
arr1 = np.array([1, 2, 2, 3, 4, 4, 5])
```

```
unique_vals = np.unique(arr1)
```

30. Replace all NaN values in a NumPy array with the mean of the array.

```
arr1= np.array([1, 2, np.nan, 4, np.nan])
```

```
mean_val = np.nanmean(arr1)
```

```
arr1 [np.isnan(arr30)] = mean_val
```